

Implementing Agentic RAG

 Demo 2

Demo: Create a Market Research Agent with RAG and Cohere

Overview

This guide explores how to create a Market Research Agent utilizing Agentic Retrieval-Augmented Generation (RAG) techniques. It combines Cohere's robust language models with real-time web searches to produce dynamic, well-supported responses. The execution is carried out through a Python script intended for console operation.

Scenario

In a practical market intelligence context, companies must collect up-to-date, precise data from multiple sources. Conventional data pipelines that rely on static documents or obsolete datasets are no longer adequate.

This practical project illustrates how to build a real-time, adaptive market research agent that utilizes Agentic RAG to:

- Comprehend user inquiries
- Obtain current data from the internet.
- Reflect on it utilizing Cohere's LLM.
- Provide trustworthy, well-founded insights.

Problem Statement

Traditional market research tools are:

- Static
- Data-locked
- Non-adaptive to live, evolving information

How can we build an adaptive agent that:

- Uses real-time data
- Integrates reasoning capabilities
- Aligns with open-source and free-tier tooling

Approach

We build an Agentic RAG-based market research agent using:

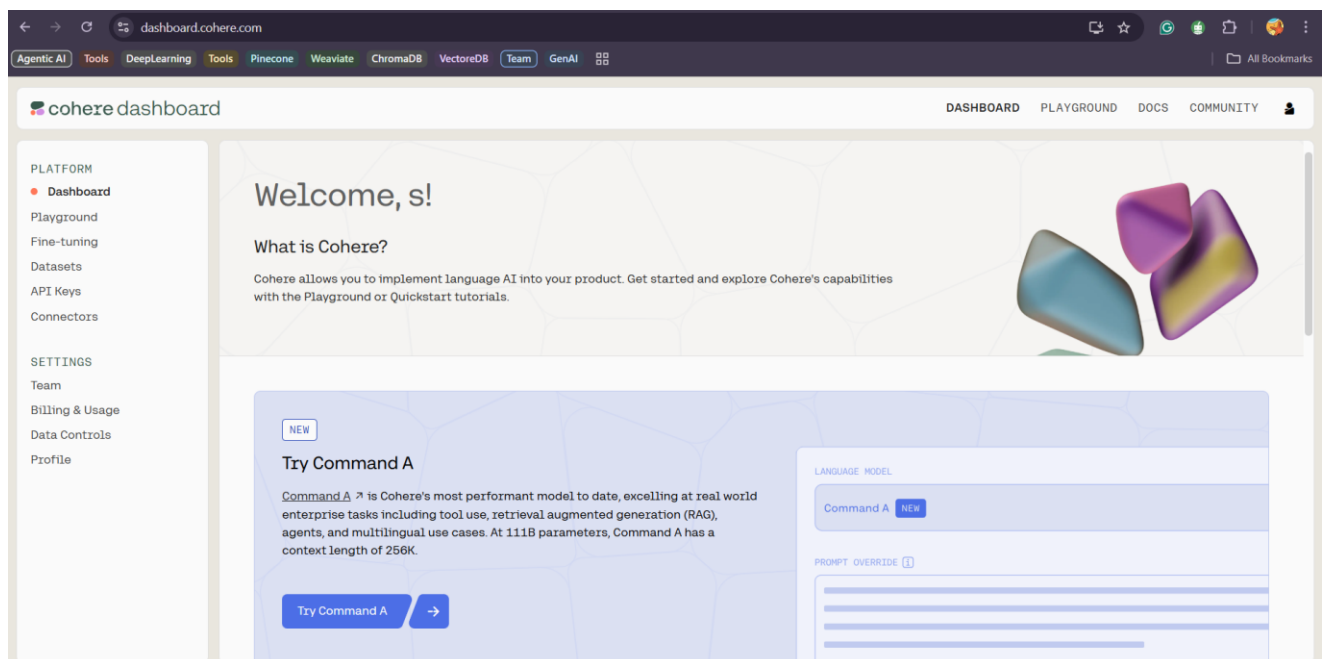
- DuckDuckGo for live web search (free)

- Cohere's command-r-plus for LLM-powered reasoning
- Modular Agentic RAG loop

This agent retrieves relevant web pages, extracts content, and lets Cohere reason over it—all through a Python console app.

Cohere API Key

<https://dashboard.cohere.com/api-keys>



Stepwise Solution

Project Structure

```
1 DEMO 2/  
2 |— market_research_agent_web.py      # Main agent script  
3 |— .env                             # Contains API keys (not committed)  
4 |— requirements.txt                  # Dependency list  
5 |— env4/                             # Python virtual environment (optional)
```

Mapping to Module: Implementing Agentic RAG

Topic	Project Coverage
Agentic RAG	Dynamic retrieval + LLM-based reasoning loop Live web + memory vs. static vector search
Architecture and Components	Multi-step agent loop with Cohere and DuckDuckGo
Adaptive RAG	On-demand, real-time web results
Variants of Agentic RAG	One-shot + loop-based agentic querying
Applications	Market intelligence use case
Agentic RAG with LlamaIndex	N/A here. Covered in Demo 1
Agentic RAG with Cohere	Yes — central to reasoning

How the Agent Works - Agentic RAG Logic

1. Receives a user query.
2. Invokes a tool (web search) to retrieve real-time documents.
3. Processes the documents and formats them.
4. Uses another tool (Cohere chat) to reason over documents.
5. Returns a grounded, natural language response.

This dynamic tool-using behavior is the heart of Agentic RAG.

Step 1: Setup Environment

- Install Python, create a virtual environment:
- Install dependencies



```
1  #Install Python, create a virtual environment:
2  python -m venv env4
3  env4\Scripts\activate
4
5  #Install dependencies:
6  pip install -r requirements.txt
7
8  # File: requirements.txt
9  cohere
10 requests
11 beautifulsoup4
12 python-dotenv
```

Step 2: Store API Key Securely

- Create .env file
- Load it in Python



```
1  # Create .env file
2  COHERE_API_KEY=your_cohere_api_key_here
3
4  # Load it in Python
5  from dotenv import load_dotenv
6  load_dotenv()
7  COHERE_API_KEY = os.getenv("COHERE_API_KEY")
```

Step 3: Perform Live Web Search (DuckDuckGo)

- We use DuckDuckGo's HTML endpoint to search the web
- We extract content from top pages

```
1 DUCKDUCKGO_SEARCH_URL = "https://html.duckduckgo.com/html/"
2
3 resp = requests.post(DUCKDUCKGO_SEARCH_URL, data={"q": query}, headers=HEADERS)
4 soup = BeautifulSoup(resp.text, "html.parser")
5 links = soup.find_all("a", class_="result__a", limit=max_results)
6 page = requests.get(href)
7 text = BeautifulSoup(page.text, "html.parser").get_text()
```

Step 4: Processing Document

- Format Documents for Cohere
- Send them to Cohere for chat-based reasoning

```
1 formatted_docs = [{"text": doc} for doc in web_docs] # list of strings
2 response = co.chat(
3     model="command-r-plus",
4     message=query,
5     documents=formatted_docs,
6 )
```

Step 5: Reasoning via LLM Tool (Cohere)

- Use the second tool: Cohere's Command R model for reasoning over documents.
- The agent sends the query along with documents, and Cohere returns a grounded answer.



```
1 response = co.chat(  
2     model="command-r-plus",  
3     message=query,  
4     documents=formatted_docs,  
5 )
```

Step 6: Build Agent Loop



```
1 while True:  
2     query = input("Enter query or 'exit': ")  
3     if query.lower() == "exit": break  
4     web_docs = web_search_duckduckgo(query)  
5     formatted_docs = [{"text": doc} for doc in web_docs]  
6     response = co.chat(  
7         model="command-r-plus",  
8         message=query,  
9         documents=formatted_docs,  
10    )  
11    print("\nAnswer:\n", response.text)
```

Terminal Output:

```

👤 Enter your market research question (or 'exit'): What are the latest AI trends in retail?

[INFO] Searching web...
[+] Retrieved: https://www.forbes.com/sites/bernardmarr/2024/10/29/the-future-of-retail-10-game-changing-trends-that-will-define-2025/
[+] Retrieved: https://blogs.nvidia.com/blog/ai-in-retail-survey-2024/
[+] Retrieved: https://www.matellio.com/blog/ai-retail-trends/
[INFO] Sending to Cohere for RAG...

🤖 Agentic RAG Answer:
The retail industry is undergoing a significant transformation driven by the rise in AI. AI is being used to enhance operational efficiency, improve customer and employee experiences, and drive growth. Here are some of the latest AI trends in retail:

- AI-powered analytics: Retailers are leveraging AI and analytics to gain insights from data and make better decisions.
- Personalization: AI-powered personalization engines can tailor product recommendations, content, and marketing messages to individual customers, improving their shopping experience.
- Computer Vision and Image Recognition: These technologies enable visual search, allowing customers to search for products by uploading an image and finding similar items.
- Chatbots and Virtual Assistants: AI-powered chatbots and virtual assistants can provide customers with 24/7 assistance, handle common queries, and improve customer service.
- Predictive Analytics: Retailers can use AI to forecast demand, optimize inventory levels, and improve supply chain efficiency.
- Natural Language Processing (NLP): NLP enables voice-based shopping, allowing customers to shop via voice assistants and smart speakers.

```

Running the Agent with Gradio:

It will start a local Gradio web interface. In the console, you'll see a URL (usually `http://127.0.0.1:7860`), and you can open it in your web browser.

```

[notice] To update, run: python.exe -m pip install --upgrade pip

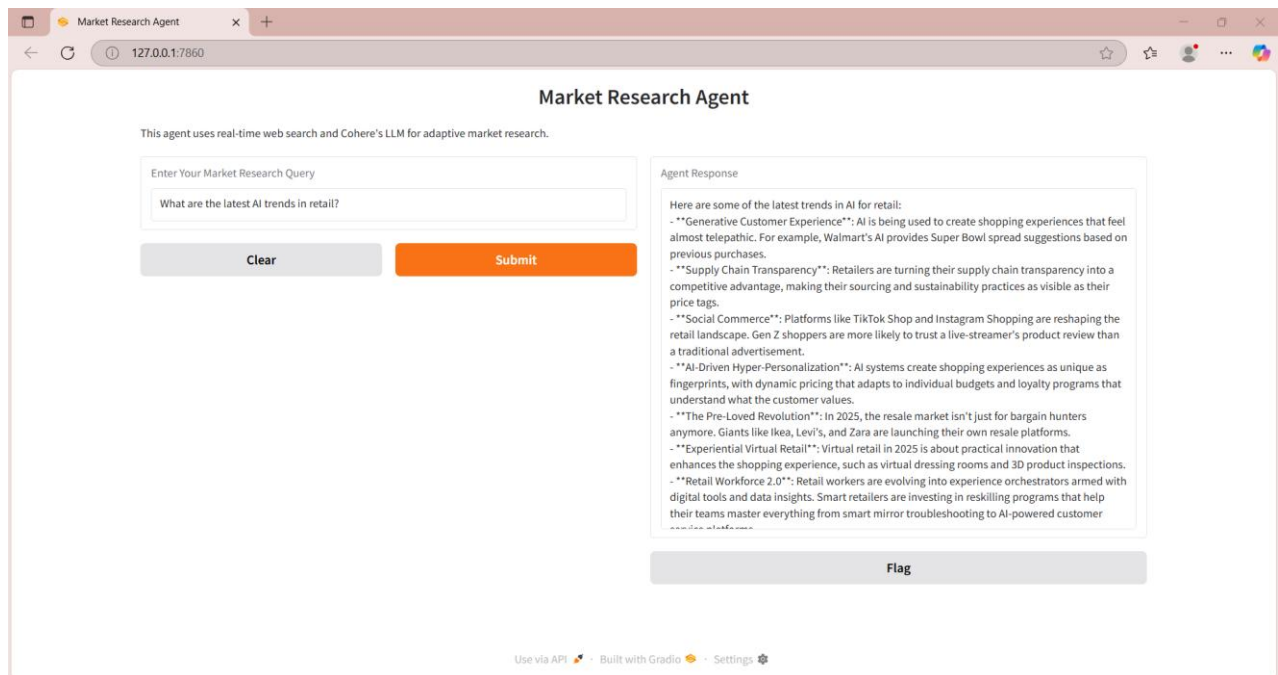
(env4) D:\Agentic AI Course\Module 5\DEMO\demo 2>python market_research_agent_web.py
* Running on local URL:  http://127.0.0.1:7860
* To create a public link, set `share=True` in `launch()`.

```

The screenshot shows a web browser window with the title "Market Research Agent". The address bar shows the URL "127.0.0.1:7860". The page content includes a header "Market Research Agent" and a sub-header "This agent uses real-time web search and Cohere's LLM for adaptive market research." Below this, there is a form with two input fields: "Enter Your Market Research Query" and "Agent Response". There are three buttons: "Clear", "Submit", and "Flag". At the bottom of the page, there is a footer that says "Use via API" with a lightning bolt icon, "Built with Gradio" with a Gradio logo, and "Settings" with a gear icon.

Example UI Behavior

- **Input Box:** The user enters a query like "What are the latest trends in AI for retail?"
- **Agent Response:** The agent performs a web search, sends the retrieved data to Cohere's LLM for reasoning, and displays the resulting response in a text box.



Optional Customizations

- **Styling:** You can enhance the Gradio UI with CSS or customize the layout further using Gradio's built-in options.
- **Error Handling:** Add error handling to manage network issues or invalid responses.
- **Webhooks:** If you want to extend this to an external platform, Gradio allows you to deploy the UI on the web or integrate with other APIs.

Summary

You now have a fully working, dynamic market research agent using:

- Real-time web data
- Adaptive agentic RAG loop
- Cohere's LLM capabilities